Table 92. TMDS_POP_DOUBLE_L1 Register

Bits	Description	Type	Reset
31:0	Get lane 1 of the encoding of two pixels' worth of colour data. Two 10-bit TMDS symbols are packed at the bottom of a 32-bit word. The POP alias shifts the colour register when read, according to the values of PIX_SHIFT and PIX2_NOSHIFT.	RF	0x00000000

SIO: TMDS_PEEK_DOUBLE_L2 Register

Offset: 0x1e0

Table 93. TMDS_PEEK_DOUBLE_L2 Register

Bits	Description	Type	Reset
31:0	Get lane 2 of the encoding of two pixels' worth of colour data. Two 10-bit TMDS symbols are packed at the bottom of a 32-bit word. The PEEK alias does not shift the colour register when read, but still advances the lane 2 DC balance state. This is useful if all 3 lanes' worth of encode are to be read at once, rather than processing the entire scanline for one lane before moving to the next lane.	RF	0x00000000

SIO: TMDS_POP_DOUBLE_L2 Register

Offset: 0x1e4

Table 94. TMDS_POP_DOUBLE_L2 Register

Bits	Description	Type	Reset
31:0	Get lane 2 of the encoding of two pixels' worth of colour data. Two 10-bit TMDS symbols are packed at the bottom of a 32-bit word. The POP alias shifts the colour register when read, according to the values of PIX_SHIFT and PIX2_NOSHIFT.	RF	0x00000000

3.2. Interrupts

Each core is equipped with an internal interrupt controller, with 52 interrupt inputs. For the most part each core has exactly the same interrupts routed to it, though there are some exceptions, referred to as *core-local interrupts*, where there is an individual per-core interrupt source mapped to the same interrupt number on each core:

- Cross-core FIFO interrupts: `SIO_IRQ_FIFO` and `SIO_IRQ_FIFO_NS` (Section 3.1.5)
- Cross-core doorbell interrupts: `SIO_IRQ_BELL` and `SIO_IRQ_BELL_NS` (Section 3.1.6)
- RISC-V platform timer (also usable by Arm cores): `SIO_IRQ_MTIMECMP` (Section 3.1.8)
- GPIO interrupts: `IO_IRQ_BANK0`, `IRQ_IO_BANK0_NS`, `IO_IRQ_QSPI`, `IO_IRQ_QSPI_NS` (Section 9.5)

The remaining interrupt inputs have the same interrupt source mirrored identically on both cores. Non-core-local interrupts should only be enabled in the interrupt controller of a single core at a time, and will be serviced by the core whose interrupt controller they are enabled in.

Table 95. System-level interrupt numbering. All interrupts are routed to both processors.

IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source
0	`TIMER0_IRQ_0`	11	`DMA_IRQ_1`	22	`IO_IRQ_BANK0_NS`	33	`UART0_IRQ`	44	`POWMAN_IRQ_POW`
1	`TIMER0_IRQ_1`	12	`DMA_IRQ_2`	23	`IO_IRQ_QSPI`	34	`UART1_IRQ`	45	`POWMAN_IRQ_TIMER`
2	`TIMER0_IRQ_2`	13	`DMA_IRQ_3`	24	`IO_IRQ_QSPI_NS`	35	`ADC_IRQ_FIFO`	46	`SPAREIRQ_IRQ_0`

IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source
3	TIMER0_IRQ_3	14	USBCTRL_IRQ	25	SIO_IRQ_FIFO	36	I2C0_IRQ	47	SPAREIRQ_IRQ_1
4	TIMER1_IRQ_0	15	PIO0_IRQ_0	26	SIO_IRQ_BELL	37	I2C1_IRQ	48	SPAREIRQ_IRQ_2
5	TIMER1_IRQ_1	16	PIO0_IRQ_1	27	SIO_IRQ_FIFO_NS	38	OTP_IRQ	49	SPAREIRQ_IRQ_3
6	TIMER1_IRQ_2	17	PIO1_IRQ_0	28	SIO_IRQ_BELL_NS	39	TRNG_IRQ	50	SPAREIRQ_IRQ_4
7	TIMER1_IRQ_3	18	PIO1_IRQ_1	29	SIO_IRQ_MTIMECMP	40	PROC0_IRQ_CTI	51	SPAREIRQ_IRQ_5
8	PWM_IRQ_WRAP_0	19	PIO2_IRQ_0	30	CLOCKS_IRQ	41	PROC1_IRQ_CTI		
9	PWM_IRQ_WRAP_1	20	PIO2_IRQ_1	31	SPI0_IRQ	42	PLL_SYS_IRQ		
10	DMA_IRQ_0	21	IO_IRQ_BANK0	32	SPI1_IRQ	43	PLL_USB_IRQ		

On RP2350, only the lower 46 IRQ signals are connected to system-level interrupt sources, and IRQs 46 to 51 are hardwired to zero (never firing). These six spare interrupts, referred to as SPAREIRQ_IRQ_0 through SPAREIRQ_IRQ_5 in the table, are deliberately reserved for the cores to interrupt themselves (via the Arm NVIC_ISPR0 registers or the Hazard3 MEIFA CSR), for example, when an interrupt handler wants to schedule a "bottom half" handler for work that must be done after exiting the interrupt handler, but before returning to the code running in the foreground.

Nested interrupts are supported in hardware: a lower-priority interrupt can be pre-empted by a higher-priority interrupt or fault, and will resume once the higher-priority handler returns. The pre-emption priority order is determined by the interrupt priority registers starting from NVIC_IPR0 (Cortex-M33) or the MEIPRA interrupt priority array CSR (Hazard3).

When there is a choice of multiple interrupts to be entered at the same dynamic priority, the interrupt with the lowest IRQ number is chosen as a tie-breaker. The system-level IRQ numbering has been chosen to generally put higher-priority interrupts at lower IRQ numbers for this reason, though the true priority is often dependent on the specific application.

3.2.1. Non-maskable interrupt (NMI)

The system IRQ signals can be routed to the Cortex-M33 non-maskable interrupt (NMI) input, by setting the bit for that IRQ number in NMI_MASK0 or NMI_MASK1. The non-maskable interrupt ignores the processor's interrupt enable/disable state (PRIMASK), and can pre-empt any other active interrupt. NMIs are generally used for emergent circumstances that require the processor's unconditional attention, such as loss of PLL lock or power supply integrity.

The NMI mask registers are core-local, so each core can have a different combination of interrupts routed to its NMI input. The NMI mask, along with all other EPPB registers, is reset by a warm reset of that core. This avoids an issue on RP2040 where the NMI mask could be left set following a processor reset.

In addition to system-level interrupts, the non-maskable interrupt is asserted when an integrity check is failed in the redundancy coprocessor (RCP, Section 3.6.3). This behaviour cannot be disabled, but a correctly-programmed RCP does not trigger under normal voltage, frequency, and temperature conditions. Likewise, if user code does not execute any RCP instructions, the RCP will never trigger. The RCP NMI output is asserted on both cores when an integrity check fails, and is de-asserted by a warm processor reset.

3.2.2. Further reading on interrupts

This section describes the routing of system-level interrupt requests to the processor subsystem. It omits important details such as the processor's response to receiving an interrupt, and how processors choose which system-level interrupt requests to subscribe to. The following is a selection of relevant information for these topics:

- Section 3.7.2.5 describes the Cortex-M33's internal interrupt controller, the NVIC
- Register listings starting from NVIC_ISER0 describe controls for NVIC operation
- Section 3.7.4.6 is an overview of Cortex-M33 exception handling

- The Armv8-M Architecture Reference Manual describes detailed architecture rules for exception handling
- Section 3.8.4 describes standard RISC-V trap handling
- Section 3.8.4.2 describes the standard RISC-V external, timer and software interrupt requests, and how they are connected on RP2350
- Section 3.8.6.1 describes the Xh3irq interrupt controller, which provides priority-controlled interrupt support for the system-level interrupts on Hazard3
- Each peripheral has its own interrupt registers which control the assertion of its system-level interrupts listed in Table 95 — see peripheral documentation for more information

3.3. Event signals (Arm)

Using the `WFE` instruction, the Cortex-M33 can enter a sleep state until an "event" (or interrupt) takes place. It can also generate events using the `SEV` instruction. RP2350 cross-wires event signals between the two processors: an event sent by one processor will be received on the other.

NOTE

> The event flag is "sticky": if both processors send an event (`SEV`) simultaneously, then enter the sleep state (`WFE`), they will both wake immediately. This prevents the processors from getting stuck in a sleep state in this scenario.

Processors also receive an event signal from the global monitor if their reservation is lost due to a write by a different master, in accordance with Armv8-M architecture requirements.

While in a `WFE` (or `WFI`) sleep state, the processor shuts off its internal clock gates to reduce power consumption. When **both** processors are in a sleep state and the DMA is inactive, all of RP2350 can enter a sleep state, disabling clocks on unused infrastructure such as the bus fabric. The rest of RP2350 wakes automatically when either of the processors wakes. See Section 6.5.2.

3.4. Event signals (RISC-V)

The Hazard3 `h3.block` instruction halts processor execution until an unblock signal is received. The `h3.unblock` instruction sends an unblock signal to other processors. These NOP-compatible hint instructions are documented in Section 3.8.6.3.

On RP2350 the Hazard3 unblock in/out signals are cross-connected between the two processors, and each processor's unblock output is also fed back into its input. The global monitor also posts an unblock signal to each core when that core loses a reservation due to an access by another core or the system DMA.

The Hazard3 MSLEEP CSR defines how deep a sleep the processor will enter when executing a `h3.block` instruction. By default this is a simple pipeline stall, but the processor can also gate its own clock and negotiate the system-level clock wake/sleep state with the clocks block (Section 6.5.2).

The `h3.unblock` instruction is "sticky": an `h3.block` will fall through immediately if any unblock signal has been received since the last time the processor executed an `h3.block` instruction.

3.5. Debug

The Serial Wire Debug (SWD) bus provides access to hardware and software debug features including:

- Loading firmware into SRAM or external flash memory

- Control of processor execution: run/halt, step, set breakpoints, other standard debug functionality
- Access to processor architectural state
- Access to memory and memory-mapped IO via the system bus
- Configuring the CoreSight trace hardware (Arm processors only)

The SWD bus is exposed on two dedicated pins, SWCLK and SWDIO. See Table 1430 for the pin definitions for SWCLK and SWDIO, and see Table 1440 for additional information on their specifications.

A single SW-DP provides access to RP2350's debug subsystem from the external SWCLK and SWDIO pins. The DP is multidrop-capable, but use of multidrop SWD is not mandatory. All hardware in the debug subsystem, with the exception of the RP-AP, can also be accessed directly from the system bus using the self-hosted debug window starting at CORESIGHT_PERIPH_BASE.

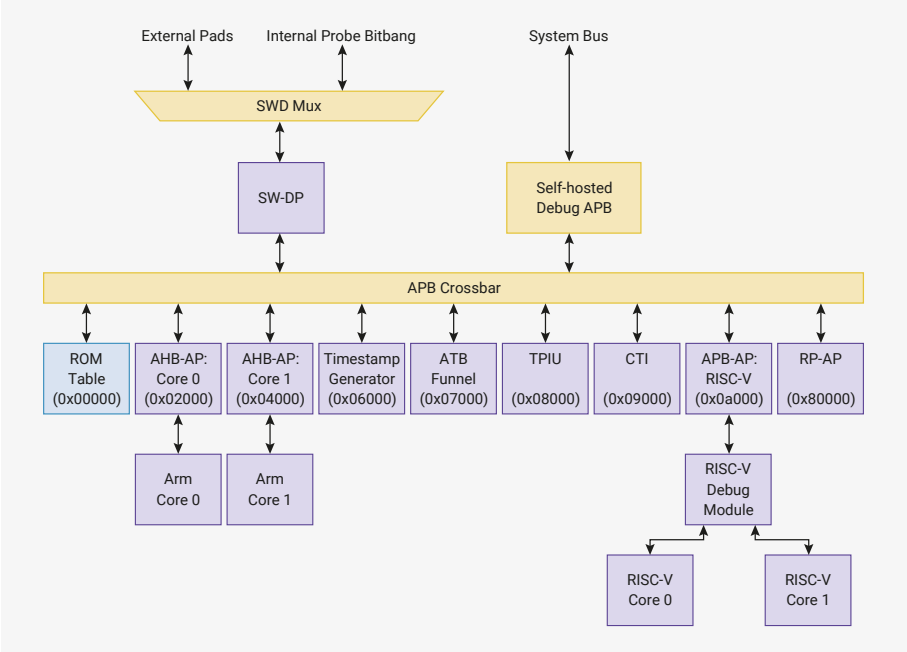


Figure 10. RP2350 debug topology. An SW-DP connects the external SWD pins to internal debug hardware. The ROM table lists debug components, for automatic discovery. AHB-APs provide debug access to Arm processors, and an APB-AP provides access to a standard RISC-V Debug Module. The RP-AP provides Raspberry-Pi-specific controls such as rescue reset and debug key entry. Remaining components are for Arm trace.

The numbers in brackets in Figure 10 are the addresses of the debug components within the debug address space. These correspond to values written to the SW-DP SELECT register for SWD accesses, or offsets from CORESIGHT_PERIPH_BASE for self-hosted debug access. All APs are accessible through the SW-DP, and all except the RP-AP are also accessible through self-hosted debug.

The SW-DP and RP-AP are in the always-on power domain, and are available once external power is applied and the power-on reset (POR) time has elapsed. All other APs in Figure 10 are available only once:

1. the power manager (POWMAN) has sequenced the first power up of the switched core domain
2. the OTP PSM has read critical hardware configuration flags from OTP
3. the system clock (`clk_sys`) is running

3.5.1. Connecting to the SW-DP

The SW-DP defaults to the Dormant state at power-up or assertion of the external reset (RUN) pin. A Dormant-to-SWD sequence must be issued before beginning SWD operations. See the Arm Debug Interface specification, version 6, for details of Dormant/SWD state switching: https://developer.arm.com/documentation/ihi0074/latest/

After a power-on, the following sequence can be used to connect to the SW-DP:

1. At least 8 × SWCLK cycles with SWDIO high.

2. The 128-bit Selection Alert sequence: `0x19bc0ea2`, `0xe3ddafe9`, `0x86852d95`, `0x6209f392`, LSB-first.
3. Four SWCLK cycles with SWDIO low.
4. SWD activation code sequence : `0x1a`, LSB first.
5. At least 50 × SWCLK cycles with SWDIO high (line reset).
6. A DPIDR read to exit the Reset state

In order to wake up the system from a low power (P1.x) state, set the CDBGPWRUPREQ in the DP CTRL/STAT register, then poll CDBGPWRUPACK in the same register until set. In low-power states, only the SW-DP and RP-AP are accessible, as the remaining debug logic is unpowered.

3.5.2. Arm debug

There are two AHB5 Mem-APs, at offsets `0x02000` and `0x04000` in the debug address space, which are used to debug the two Arm Cortex-M33 processors. Each Mem-AP is an AHB5 manager which accesses a 32-bit downstream address space. This is the same address space accessed by a processor's load/store instructions, which includes system-level hardware such as memory and peripherals, and processor-internal hardware on the processor's private peripheral bus (PPB). Certain PPB registers are visible only when accessed from the Mem-AP, not when accessed by software running on the processor.

The AHB5 Mem-AP's own register map is defined in Arm's ADIv6 specification. Generally this is only of interest to those implementing their own debug translator, and the Mem-AP can be thought of simply as a bridge between a DP (such as RP2350's SW-DP) and a downstream address space.

The standard Arm debug registers used to debug software running on the Cortex-M33 can be found documented in the Armv8-M Architecture Reference Manual, or the Cortex-M33 Technical Reference Manual, available from Arm Ltd. This datasheet also documents the core's internal registers in Section 3.7.5.

The Mem-APs can access system peripherals and memory at exactly the same addresses they would be accessed by software running on the processor. However, the privilege and security of Mem-AP accesses may be different from the security state of the software running on the processor at the point it halted: the privilege and security of Mem-AP accesses is configured explicitly via its control and status word (CSW) register. Care must be taken when debugging Non-secure software which accesses the SIO, for example, because by default the debugger may access the Secure alias of the SIO, not the Non-secure alias which software will have been accessing.

The bus filters configured by the ACCESSCTRL bus access permission registers (Section 10.6.2) treat bus accesses originating from the Mem-APs as distinct from bus accesses originating from software running on the processor. This means it is possible to lock software out from a peripheral, whilst still allowing debugger access.

3.5.3. RISC-V debug

There is a single APB Mem-AP, at offset `0x0a000` in the debug address space, which provides access only to the RISC-V Debug Module (DM). The DM is a standard component which the debugger uses to enumerate RISC-V harts present in the system, debug software running on each hart, and access the system bus. It is defined in the RISC-V debug specification, of which RP2350 implements version 0.13.2.

From the point of view of the RISC-V debug specification, the SW-DP and APB Mem-AP function jointly as the Debug Transport Module for this system. The DM is located at offset `0x0` in the APB-AP's downstream address space, and the registers are word-sized and byte-addressed, meaning the DM register addresses in the debug specification must be multiplied by 4 to get the correct APB address.

On RP2350, each core possesses exactly one hardware thread (hart). Core 0 has a hart ID of 0, and core 1 has a hart ID of 1. These hart IDs match the hart index used in the DM. This DM is also equipped with the hart array mask select extension, which allows multiple cores to be reset/halted/resumed simultaneously.

The DM is equipped with the System Bus Access (SBA) extension, which allows the debugger to access the system bus without halting either core. This can be used for minimally intrusive debug techniques like Segger RTT. SBA accesses

arbitrate with core 1's load/store port to access the system bus, but they are treated as distinct from core 1's accesses for the purpose of bus filtering (Section 10.6.2), which means it is possible to lock software out of a peripheral whilst retaining debug access. Processor load/stores in Debug mode are also treated as debug accesses for the purpose of bus filtering.

The DM is able to reset each core individually using the `dmcontrol.hartreset` control. This resets *only* the selected processor. The `dmcontrol.ndmreset` resets both processors *only*, which is the minimum requirement in the RISC-V debug specification. A full system reset, which includes the DM, can be performed using the SYSRESETREQ control in the SW-DP, a switched core domain reset configured in POWMAN and initiated by the watchdog, or any full-system reset such as the RUN pin. A PSM reset initiated by the watchdog can reset almost all system-level hardware except for the DM, but note that the DM becomes momentarily inaccessible whilst the system clock's clock generator is reset, which is the reason for `dmcontrol.ndmreset` resetting the processors only.

For details on the processor side of RISC-V debug, see Section 3.8.5. See also the Hazard3 source code at github.com/Wren6991/Hazard3, which includes the DM implementation under the `hdl/debug/dm/` directory.

3.5.4. Debug power domains

The SW-DP and the RP-AP are in the always-on power domain. This means they are available even when the system is in its lowest-power state, with the switched core domain (which includes the processors) fully powered down.

The remainder of the debug hardware is in the switched core domain. This is the same domain as the processors and system peripherals.

Setting the CDBGPWRUPREQ bit in the SW-DP's CTRL/STAT register will force a power up of the switched core domain, making the remaining debug hardware available. This power up takes some time, as it is sequenced by the 32 kHz low-power oscillator (Section 8.4), so the CDBGPWRUPACK bit must be polled to wait for the system to power up before attempting to access any APs other than the RP-AP. See Arm's ADIv6 specification for the SW-DP's register listing.

Note that the RP-AP is accessible without asserting CDBGPWRUPREQ, as it is always powered.

3.5.5. Software control of SWD pins

The DBGFORCE register in SYSCFG can be used to detach the SW-DP from the external debug pads, and instead bitbang the internal SWD signals directly from software. This is intended for a debug probe running on one core being used to debug the other core. For other use cases it is generally cleaner to use the self-hosted debug access to interface with the APs directly from the system bus.

3.5.6. Self-hosted debug

All APs shown in Figure 10, except for the RP-AP, have direct memory-mapped access from the system bus. This is known as self-hosted debug, because with care it allows running a debug host (i.e. a debugger) directly on-system. It can also be used to access the trace hardware, which can be used for self-hosted trace using the trace DMA FIFO. By default only Secure access is permitted, as the processor debug presents an opportunity for Non-secure code to interfere with the Secure context and/or perform Secure bus accesses.

The self-hosted debug window starts at address `0x40140000` (CORESIGHT_PERIPH_BASE). The offsets of the APs within this window are the same as the APs' addresses when accessed from the SW-DP.

Because of the blocking nature of the AHB-AP's DRW register, and its interactions with the Cortex-M33's arbitration of AHB-AP accesses with load/stores, certain accesses have potential to cause bus lockup due to circular bus stall dependencies. In particular, cores may not access their own AHB-APs through the self-hosted debug window, and AHB-APs may not access AHB-APs through the self-hosted debug window — attempting to do so will immediately return a bus fault. To reduce the opportunities for deadlock, a full APB crossbar is used to connect the SW-DP and the self-hosted debug port to the APs, so that for example self-hosted use of the Arm trace hardware will not interfere with an external debugger attaching via the AHB-APs.

There are some cases where a bus deadlock can not be avoided, such as a core using the other core's AHB-AP, via the self-hosted debug window, to access some other APB peripheral:

1. The access upstream of the APB's DRW register will not complete until the downstream access completes
2. The downstream access will not complete until it is granted access to the system APB bridge
3. Access to the APB bridge will not be granted until the upstream access, which is occupying the system APB bridge, completes
4. See point 1.

This situation can arise when running a self-hosted debugger on one core, and debugging code on the other core which accesses APB addresses. The deadlock is eventually broken when the APB bridge's 65536-cycle timeout expires, abandoning the transfer and returning a bus error to the origin of the upstream access. To avoid this, software should detect when it is about to use an AP to access an APB address (an address starting with `0x4`), and perform the access directly instead of using the Mem-AP.

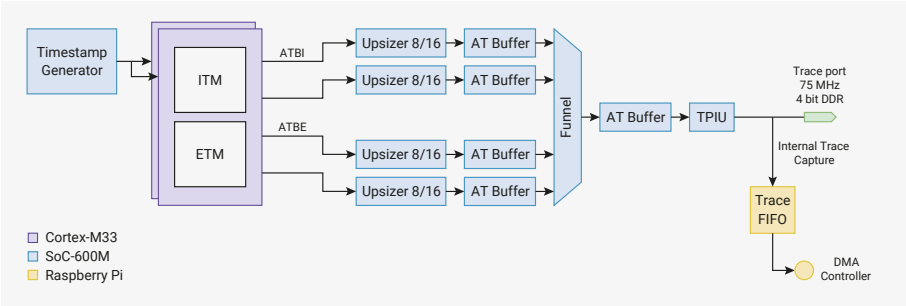
This type of deadlock does not occur when the debugger accesses the bus with RISC-V System Bus Access, because the bus transfer upstream of the DM does not block on completion of the downstream access.

3.5.7. Trace

3.5.7.1. Overview

The ATB trace subsystem is based on the Coresight SoC-600M architecture, as shown in Figure 11.

Figure 11. Trace Subsystem



The trace subsystem captures trace messages from each of the Cortex-M33 ITM/ETM components, merges them into a single trace bus, and sends off-chip through the 4-bit DDR trace port for subsequent capture and analysis by a trace port analyser.

This allows the developer to review a detailed log of software executed on the processors. The advantage over conventional hardware debug is that it does this without halting the processors or affecting their execution timing, so you can diagnose software issues that are hard to reproduce under a debugger.

The trace subsystem comprises the following main components:

- Timestamp Generator: Timestamps propagate to both Cortex-M33 processors, and are applied to ETM and ITM output so that the relative timing of their trace streams can be recovered.
- Cortex-M33 ETM: Embedded Trace Macrocell, for real-time instruction flow messages generated from observations of the Cortex-M33's execution.
- Cortex-M33 ITM: Instruction Trace Macrocell, for software-generated messages.
- ATB Funnel: Merges the Cortex-M33 trace sources into a single trace stream using the timestamps from the Timestamp Generator.
- TPIU: Trace Port Interface Unit, outputs trace data over trace port pins. The source-synchronous trace interface is 4-bits DDR, up to 75 MHz clock, giving a maximum trace data rate of up to 600 Mb/s.

- Trace FIFO: Optionally captures the 32-bit TPIU trace stream on-device, from which point the DMA can transfer to main system SRAM.

See the Arm CoreSight ETM-M33 Technical Reference Manual for information about the Cortex-M33 ETM. See the SoC-600M Technical Reference Manual for information about the other trace components in Figure 11

The trace output clock is fixed at one half of `clk_sys`. At the maximum system frequency of 150 MHz this yields a 75 MHz TPIU output clock. The trace throughput is reduced at lower system clock frequencies, though this is rarely an issue in practice as the processor instruction throughput (and therefore the demand for trace output bandwidth) scales accordingly.

3.5.7.2. Trace FIFO

Trace output goes to one of two data sinks:

- The four-bit TPIU interface streams data out of the chip through GPIOs, for capture by an external probe
- The trace FIFO streams data into SRAM via the system DMA

The bandwidth of the DMA is greater than the bandwidth of the TPIU interface. Capturing into an on-chip buffer also allows trace to operate through a comparatively low-speed SWD probe without restricting trace bandwidth.

The operation is similar to a micro-trace buffer (MTB). However, all of system SRAM is available for trace. You can also use other DMA endpoints like the PIO and HSTX to implement your own trace data sinks, for example if you would prefer a wider and lower-frequency bus than the TPIU provides.

You must enable DMA access to the trace FIFO registers by setting the `DMA` bit in the ACCESSCTRL CORESIGHT_TRACE register before attempting to DMA from this FIFO. Configure the DMA for DREQ `53` to select the trace FIFO.

3.5.7.3. List of trace FIFO registers

The trace FIFO registers start at a base address of `0x50700000` (defined as CORESIGHT_TRACE_BASE in the SDK).

Table 96. List of CORESIGHT_TRACE registers

Offset	Name	Info
0x0	CTRL_STATUS	Control and status register
0x4	TRACE_CAPTURE_FIFO	FIFO for trace data captured from the TPIU

CORESIGHT_TRACE: CTRL_STATUS Register

Offset: 0x0

Description

Control and status register

Table 97. CTRL_STATUS Register

Bits	Description	Type	Reset
31:2	Reserved.	-	-
1	**TRACE_CAPTURE_FIFO_OVERFLOW**: This status flag is set high when trace data has been dropped due to the FIFO being full at the point trace data was sampled. Write 1 to acknowledge and clear the bit.	RW	0x0

Bits	Description	Type	Reset
0	**TRACE_CAPTURE_FIFO_FLUSH**: Set to 1 to continuously hold the trace FIFO in a flushed state and prevent overflow. Before clearing this flag, configure and start a DMA channel with the correct DREQ for the TRACE_CAPTURE_FIFO register. Clear this flag to begin sampling trace data, and set once again once the trace capture buffer is full. You must configure the TPIU in order to generate trace packets to be captured, as well as components like the ETM further upstream to generate the event stream propagated to the TPIU.	RW	0x1

CORESIGHT_TRACE: TRACE_CAPTURE_FIFO Register

Offset: 0x4

Description

FIFO for trace data captured from the TPIU

Table 98. TRACE_CAPTURE_FIFO Register

Bits	Description	Type	Reset
31:0	**RDATA**: Read from an 8 x 32-bit FIFO containing trace data captured from the TPIU. Hardware pushes to the FIFO on rising edges of clk_sys, when either of the following is true: * TPIU TRACECTL output is low (normal trace data) * TPIU TRACETCL output is high, and TPIU TRACEDATA0 and TRACEDATA1 are both low (trigger packet) These conditions are in accordance with Arm Coresight Architecture Spec v3.0 section D3.3.3: Decoding requirements for Trace Capture Devices The data captured into the FIFO is the full 32-bit TRACEDATA bus output by the TPIU. Note that the TPIU is a DDR output at half of clk_sys, therefore this interface can capture the full 32-bit TPIU DDR output bandwidth as it samples once per active edge of the TPIU output clock.	RF	0x00000000

3.5.8. Rescue reset

A rescue reset is a full system reset, similar to asserting the RUN pin low, which also sets a flag telling the bootrom to halt before running any user software. This is performed over the SWD bus using the RP-AP, and can be performed even when system clocks are stopped and the switched core power domain is powered down. This is used in the case where the chip has locked up, for example if code has been programmed into flash which permanently halts the system clock: since the debugger can no longer communicate with the processors to return the system to a working state, more drastic action is needed. This functionality was provided by the Rescue DP on RP2040, but on RP2350 it is provided by the RP-AP, to avoid mandatory use of multidrop SWD.

A rescue is invoked by setting and then clearing the CTRL.RESCUE_RESTART bit in the RP-AP. This causes a hard reset of the chip, and sets CHIP_RESET.RESCUE_FLAG to indicate that a rescue reset took place. The bootrom checks this flag almost immediately in the initial boot process (before watchdog, flash or USB boot), acknowledges by clearing the bit, then halts the processor. This leaves the system in a safe state, with the system clock running, so that the debugger can reattach to the cores and load fresh code.

3.5.9. Security

By default, the SWD debug access port allows an external debugger to access all system memory and peripherals, and to observe and change the execution of software running on the processors. If boot signature enforcement is enabled (Section 10.1.1), debug access becomes a security concern, as it is able to sidestep this protection. To account for this, RP2350 supports progressively locking down the debug port using configuration in on-chip OTP storage.

Conceptually there are two control bits: debug disable, and secure debug disable. Debug disable is intended to completely cut off debug access to the processors and the system bus, whilst the secure debug disable forbids Secure bus accesses, and halting of processors in the Secure state, but still allows Non-secure software to be debugged as normal. There are two ways to set these control bits:

- Setting the relevant OTP critical flag: CRIT1.DEBUG_DISABLE or CRIT1.SECURE_DEBUG_DISABLE to set the debug disable or secure debug disable, respectively
- Installing a 128-bit fixed debug key as OTP key 5 or 6 (Section 3.5.9.2)

OTP configuration changes take effect at the next reset of the OTP block.

Once debug has been disabled, software can re-enable debug using the OTP DEBUGEN register, which allows the secure and overall debug enable to be cleared individually for each processor. For example, Secure software may implement a shell where users can authenticate using a cryptographic challenge to enable debug on systems where it is disabled by default. The DEBUGEN register belongs to the processor cold reset domain, so it is preserved over a PSM reset starting from as early as OTP (the second PSM stage). This allows almost a full system reset without losing debug access.

To avoid accidental writes of the DEBUGEN register, its bits can be individually locked using the matching bits in DEBUGEN_LOCK.

This offers increasing levels of debug protection:

1. Fully open: no keys installed and no OTP debug disable flags are set. This is the most convenient configuration for product development.
2. Access with key only: at least one key is installed, but no OTP debug disable flags are set.
3. No access even with key (an OTP debug disable flag is set), but Secure code can enable debug access by writing to DEBUGEN.
4. No access even with key (an OTP debug disable flag is set), and DEBUGEN is locked by DEBUGEN_LOCK.

3.5.9.1. Effects of debug disables

The secure debug disable flag (CRIT1.SECURE_DEBUG_DISABLE) has the following effects:

- Set Secure AP enable signals for Arm core 0 and core 1 AHB-APs to `0`.
  - This prevents the APs from performing Secure bus accesses (including to the PPB).
  - Status is reported in the `SDeviceEn` flag of the AHB-AP `CSW` register.
- Set the Cortex-M33 `SPIDEN` and `SPNIDEN` signals for both cores to `0`.
  - This prevents the cores from being halted or traced whilst in the Secure state.
- Disable the factory test JTAG interface (Section 10.10).

NOTE

Both AHB-APs' `CSW.HNONSEC` bits default to 0, generating Secure bus accesses. If the secure debug disable flag is set, these bits must be set to 1 to generate Non-Secure bus accesses.

The debug disable flag (CRIT1.DEBUG_DISABLE) has all of the effects of the secure debug disable flag. It also has the following additional effects:

- Set AP enable signals for Arm core 0 and core 1 AHB-APs to 0.
  - This prevents the APs from performing any bus accesses at all (including to the PPB).
  - Status is reported in the `DeviceEn` flag of the AHB-AP `CSW` register.
- Set AP enable signal for RISC-V DM APB-AP to 0.
  - This prevents the AP from accessing the RISC-V Debug Module.
  - Status is reported in the `DeviceEn` flag of the APB-AP `CSW` register.
- Set `DBGEN` and `NIDEN` signals for the CTI to 0.

On RISC-V CRIT1.SECURE_DEBUG_DISABLE has no useful effect. Debug-mode accesses from the cores always have Secure and Privileged bus attributes, except when reduced by FORCE_CORE_NS. Likewise, System Bus Access via the Debug Module is always Secure and Privileged, unless FORCE_CORE_NS.CORE1 is set, in which case it is Non-secure and Privileged. Use the CRIT1.DEBUG_DISABLE flag on RISC-V.

3.5.9.2. Debug keys

Section 13.5.2 describes the OTP hardware access keys. Hardware reads OTP access keys into hidden registers as part of the OTP power-up sequence which takes place after an OTP reset, and the corresponding OTP locations then become inaccessible. OTP keys `5` and `6` are special in that they control access to the SWD debug hardware in addition to functioning as normal OTP page keys.

A debug key is a 128-bit fixed challenge. Installing a debug key in OTP locks down debug access, and it remains locked until the debug host writes a matching key value through the RP-AP DBGKEY register. This is a write-only interface.

To install a debug key, first program the OTP locations starting from KEY5_0 or KEY6_0. These locations are ECC-protected. Once you have programmed the 128-bit key value and read it back to confirm the correct value is programmed, write the raw bit pattern `0x010101` to KEY5_VALID or KEY6_VALID to mark the key as valid. The validity takes effect at the next reset of the OTP block.

Once a key is valid, the OTP storage locations for that key become inaccessible for both reads and writes. Only the OTP power-up state machine (Section 13.3.4) can read the key.

The effect of installing debug keys depends on which of key `5` and `6` are installed:

- If key 5 or key 6 is valid, and no matching key (either) has been entered through the RP-AP, all debug is disabled. This has the same effect as setting CRIT1.DEBUG_DISABLE.
- If key 5 is valid, and no matching key (key 5 specifically) has been entered through the RP-AP, Secure debug is disabled. This has the same effect as writing CRIT1.SECURE_DEBUG_DISABLE.

When both keys are installed, key `5` provides both Secure and Non-secure debug access, and key `6` provides Non-secure debug access only. When only a single key is installed, that key provides both Secure and Non-secure debug access.

To enter a key over SWD, first write a `1` to DBGKEY.RESET. Then sequentially write 128 bits to DBGKEY.DATA, each accompanied by a `1` written to DBGKEY.PUSH. Write the data LSB-first, starting with the lowest-numbered OTP row.

Assuming you wrote a value that matched one of the installed debug keys, debug unlocks after the 128th push. The `SDeviceEn` and `DeviceEn` flags in the Mem-AP `CSW` registers indicate success or failure.

Failure to supply a matching key through the RP-AP disables debug if it would otherwise be enabled. However, supplying a key does not enable if it is already disabled for other reasons. For example, if CRIT1.DEBUG_DISABLE is set, and

DEBUGEN is clear, debug is be disabled no matter the state of the debug keys and the RP-AP.

3.5.10. RP-AP

The RP-AP is a small register block which is always accessible over SWD. RP-AP access does not require the switched core domain to be powered up, or any internal system clock generators to be running.

3.5.10.1. List of registers

The RP-AP registers start at offset `0x80000` in the debug address space, which is accessed via address `0x80000` in the SW-DP's SELECT register. Unlike the other APs, it can not be accessed directly from the system bus.

Table 99. List of RP_AP registers

Offset	Name	Info
0x000	CTRL	This register is primarily used for DFT but can also be used to overcome some power up problems. However, it should not be used to force power up of domains. Use DBG_POW_OVRD for that.
0x004	DBGKEY	Serial key load interface (write-only)
0x008	DBG_POW_STATE_SWCORE	This register indicates the state of the power sequencer for the switched-core domain. The sequencer timing is managed by the POWMAN_SEQ_* registers. See the header file for those registers for more information on the timing. Power up of the domain commences by clearing bit 0 (IS_PD) then bits 1-8 are set in sequence. Bit 8 (IS_PU) indicates the sequence is complete. Power down of the domain commences by clearing bit 8 (IS_PU) then bits 7-1 are cleared in sequence. Bit 0 (IS_PU) is then set to indicate the sequence is complete. Bits 9-11 describe the states of the power manager clocks which change as clock generators in the switched-core become available following switched-core power up. This bus can be sent to GPIO for debug. See DBG_POW_OUTPUT_TO_GPIO in the DBG_POW_OVRD register.
0x00c	DBG_POW_STATE_XIP	This register indicates the state of the power sequencer for the XIP domain. The sequencer timing is managed by the POWMAN_SEQ_* registers. See the header file for those registers for more information on the timing. Power up of the domain commences by clearing bit 0 (IS_PD) then bits 1-8 are set in sequence. Bit 8 (IS_PU) indicates the sequence is complete. Power down of the domain commences by clearing bit 8 (IS_PU) then bits 7-1 are cleared in sequence. Bit 0 (IS_PU) is then set to indicate the sequence is complete.

Offset	Name	Info
0x010	DBG_POW_STATE_SRAM0	This register indicates the state of the power sequencer for the SRAM0 domain. The sequencer timing is managed by the POWMAN_SEQ_* registers. See the header file for those registers for more information on the timing. Power up of the domain commences by clearing bit 0 (IS_PD) then bits 1-8 are set in sequence. Bit 8 (IS_PU) indicates the sequence is complete. Power down of the domain commences by clearing bit 8 (IS_PU) then bits 7-1 are cleared in sequence. Bit 0 (IS_PU) is then set to indicate the sequence is complete.
0x014	DBG_POW_STATE_SRAM1	This register indicates the state of the power sequencer for the SRAM1 domain. The sequencer timing is managed by the POWMAN_SEQ_* registers. See the header file for those registers for more information on the timing. Power up of the domain commences by clearing bit 0 (IS_PD) then bits 1-8 are set in sequence. Bit 8 (IS_PU) indicates the sequence is complete. Power down of the domain commences by clearing bit 8 (IS_PU) then bits 7-1 are cleared in sequence. Bit 0 (IS_PU) is then set to indicate the sequence is complete.
0x018	DBG_POW_OVRD	This register allows external control of the power sequencer outputs for all the switched power domains. If any of the power sequencers stall at any stage then force power up operation of all domains by running this sequence: - set DBG_POW_OVRD = 0x3b to force small power switches on, large power switches off, resets on and isolation on - allow time for the domain power supplies to reach full rail - set DBG_POW_OVRD = 0x3b to force large power switches on - set DBG_POW_OVRD = 0x37 to remove isolation - set DBG_POW_OVRD = 0x17 to remove resets
0x01c	DBG_POW_OUTPUT_TO_GPIO	Send some, or all, bits of DBG_POW_STATE_SWCORE to gpios. Bit 0 sends bit 0 of DBG_POW_STATE_SWCORE to GPIO 34 Bit 1 sends bit 1 of DBG_POW_STATE_SWCORE to GPIO 35 Bit 2 sends bit 2 of DBG_POW_STATE_SWCORE to GPIO 36 . . Bit 11 sends bit 11 of DBG_POW_STATE_SWCORE to GPIO 45
0xdfc	IDR	Standard Coresight ID Register

RP_AP: CTRL Register

Offset: 0x000

Description

This register is primarily used for DFT but can also be used to overcome some power up problems. However, it should not be used to force power up of domains. Use DBG_POW_OVRD for that.

Table 100. CTRL Register

Bits	Description	Type	Reset
31	**RESCUE_RESTART**: Allows debug of boot problems by restarting the chip with minimal boot code execution. Write to 1 to put the chip in reset then write to 0 to restart the chip with the rescue flag set. The rescue flag is in the POWMAN_CHIP_RESET register and is read by boot code. The rescue flag is cleared by writing 0 to POWMAN_CHIP_RESET_RESCUE_FLAG or by resetting the chip by any means other than RESCUE_RESTART.	RW	0x0
30	**SPARE**: Unused	RW	0x0
29:7	Reserved.	-	-
6	**DBG_FRCE_GPIO_LPCK**: Allows chip start-up when the Low Power Oscillator (LPOSC) is inoperative or malfunctioning and also allows the initial power sequencing rate to be adjusted. Write to 1 to force the LPOSC output to be driven from a GPIO (gpio20 on 80-pin package, gpio34 on the 60-pin package). If the LPOSC is inoperative or malfunctioning it may also be necessary to set the LPOSC_STABLE_FRCE bit in this register. The user must provide a clock on the GPIO. For normal operation use a clock running at around 32kHz. Adjusting the frequency will speed up or slow down the initial power-up sequence.	RW	0x0
5	**LPOSC_STABLE_FRCE**: Allows the chip to start-up even though the Low Power Oscillator (LPOSC) is failing to set its stable flag. Initial power sequencing is clocked by LPOSC at around 32kHz but does not start until the LPOSC declares itself to be stable. If the LPOSC is otherwise working correctly the chip will boot when this bit is set. If the LPOSC is not working then DBG_FRCE_GPIO_LPCK must be set and an external clock provided.	RW	0x0
4	**POWMAN_DFT_ISO_OFF**: Holds the isolation gates between power domains in the open state. This is intended to hold the gates open for DFT and power manager debug. It is not intended to force the isolation gates open. Use the overrides in DBG_POW_OVRD to force the isolation gates open or closed.	RW	0x0
3	**POWMAN_DFT_PWRON**: Holds the power switches on for all domains. This is intended to keep the power on for DFT and debug, rather than for switching the power on. The power switches are not sequenced and the sudden demand for current could cause the always-on power domain to brown out. This register is in the always-on domain therefore chaos could ensue. It is recommended to use the DBG_POW_OVRD controls instead.	RW	0x0
2	**POWMAN_DBGMODE**: This prevents the power manager from powering down and resetting the switched-core power domain. It is intended for DFT and for debugging the power manager after the chip has booted. It cannot be used to force initial power on because it simultaneously deasserts the reset.	RW	0x0
1	**JTAG_FUNCSEL**: Multiplexes the JTAG ports onto GPIO0-3	RW	0x0
0	**JTAG_TRSTN**: Resets the JTAG module. Active low.	RW	0x0

RP_AP: DBGKEY Register

Offset: 0x004

Description

Serial key load interface (write-only)

Table 101. DBGKEY Register

Bits	Description	Type	Reset
31:3	Reserved.	-	-

Bits	Description	Type	Reset
2	**RESET**: Reset (before sending a new key)	RW	0x0
1	**PUSH**	RW	0x0
0	**DATA**	RW	0x0

RP_AP: DBG_POW_STATE_SWCORE Register

Offset: 0x008

Description

This register indicates the state of the power sequencer for the switched-core domain.
The sequencer timing is managed by the POWMAN_SEQ_* registers. See the header file for those registers for more information on the timing.
Power up of the domain commences by clearing bit 0 (IS_PD) then bits 1-8 are set in sequence. Bit 8 (IS_PU) indicates the sequence is complete.
Power down of the domain commences by clearing bit 8 (IS_PU) then bits 7-1 are cleared in sequence. Bit 0 (IS_PU) is then set to indicate the sequence is complete.
Bits 9-11 describe the states of the power manager clocks which change as clock generators in the switched-core become available following switched-core power up.
This bus can be sent to GPIO for debug. See DBG_POW_OUTPUT_TO_GPIO in the DBG_POW_OVRD register.

Table 102. DBG_POW_STATE_SWCORE Register

Bits	Description	Type	Reset
31:12	Reserved.	-	-
11	**USING_FAST_POWCK**: Indicates the source of the power manager clock. On switched-core power up the clock switches from the LPOSC to clk_ref and this flag will be set. clk_ref will be running from the ROSC initially but will switch to XOSC when it comes available. On switched-core power down the clock switches to LPOSC and this flag will be cleared.	RO	0x0
10	**WAITING_POWCK**: Indicates the switched-core power sequencer is waiting for the power manager clock to update. On switched-core power up the clock switches from the LPOSC to clk_ref. clk_ref will be running from the ROSC initially but will switch to XOSC when it comes available. On switched-core power down the clock switches to LPOSC. If the switched-core power up sequence stalls with this flag active then it means clk_ref is not running which indicates a problem with the ROSC. If that happens then set DBG_POW_RESTART_FROM_XOSC in the DBG_POW_OVRD register to avoid using the ROSC. If the switched-core power down sequence stalls with this flag active then it means LPOSC is not running. The solution is to not stop LPOSC when the switched-core power domain is powered.	RO	0x0
9	**WAITING_TIMCK**: Indicates that the switched-core power sequencer is waiting for the AON-Timer to update. On switched-core power-up there is nothing to be done. The AON-Timer continues to run from the LPOSC so this flag will not be set. Software decides whether to switch the AON-Timer clock to XOSC (via clk_ref). On switched-core power-down the sequencer will switch the AON-Timer back to LPOSC if software switched it to XOSC. During the switchover the WAITING_TIMCK flag will be set. If the switched-core power down sequence stalls with this flag active then the only recourse is to reset the chip and change software to not select XOSC as the AON-Timer source.	RO	0x0
8	**IS_PU**: Indicates the power somain is fully powered up.	RO	0x0
7	**RESET_FROM_SEQ**: Indicates the state of the reset to the power domain.	RO	0x0

Bits	Description	Type	Reset
6	**ENAB_ACK**: Indicates the state of the enable to the power domain.	RO	0x0
5	**ISOLATE_FROM_SEQ**: Indicates the state of the isolation control to the power domain.	RO	0x0
4	**LARGE_ACK**: Indicates the state of the large power switches for the power domain.	RO	0x0
3	**SMALL_ACK2**: The small switches are split into 3 chains. In the power up sequence they are switched on separately to allow management of the VDD rise time. In the power down sequence they switch off simultaneously with the large power switches. This bit indicates the state of the last element in small power switch chain 2.	RO	0x0
2	**SMALL_ACK1**: This bit indicates the state of the last element in small power switch chain 1.	RO	0x0
1	**SMALL_ACK0**: This bit indicates the state of the last element in small power switch chain 0.	RO	0x0
0	**IS_PD**: Indicates the power somain is fully powered down.	RO	0x0

RP_AP: DBG_POW_STATE_XIP Register

Offset: 0x00c

Description

This register indicates the state of the power sequencer for the XIP domain.

The sequencer timing is managed by the POWMAN_SEQ_* registers. See the header file for those registers for more information on the timing.

Power up of the domain commences by clearing bit 0 (IS_PD) then bits 1-8 are set in sequence. Bit 8 (IS_PU) indicates the sequence is complete.

Power down of the domain commences by clearing bit 8 (IS_PU) then bits 7-1 are cleared in sequence. Bit 0 (IS_PU) is then set to indicate the sequence is complete.

Table 103. DBG_POW_STATE_XIP Register

Bits	Description	Type	Reset
31:9	Reserved.	-	-
8	**IS_PU**: Indicates the power somain is fully powered up.	RO	0x0
7	**RESET_FROM_SEQ**: Indicates the state of the reset to the power domain.	RO	0x0
6	**ENAB_ACK**: Indicates the state of the enable to the power domain.	RO	0x0
5	**ISOLATE_FROM_SEQ**: Indicates the state of the isolation control to the power domain.	RO	0x0
4	**LARGE_ACK**: Indicates the state of the large power switches for the power domain.	RO	0x0
3	**SMALL_ACK2**: The small switches are split into 3 chains. In the power up sequence they are switched on separately to allow management of the VDD rise time. In the power down sequence they switch off simultaneously with the large power switches. This bit indicates the state of the last element in small power switch chain 2.	RO	0x0
2	**SMALL_ACK1**: This bit indicates the state of the last element in small power switch chain 1.	RO	0x0
1	**SMALL_ACK0**: This bit indicates the state of the last element in small power switch chain 0.	RO	0x0

Bits	Description	Type	Reset
0	**IS_PD**: Indicates the power somain is fully powered down.	RO	0x0

RP_AP: DBG_POW_STATE_SRAM0 Register

Offset: 0x010

Description

This register indicates the state of the power sequencer for the SRAM0 domain.
The sequencer timing is managed by the POWMAN_SEQ_* registers. See the header file for those registers for more information on the timing.
Power up of the domain commences by clearing bit 0 (IS_PD) then bits 1-8 are set in sequence. Bit 8 (IS_PU) indicates the sequence is complete.
Power down of the domain commences by clearing bit 8 (IS_PU) then bits 7-1 are cleared in sequence. Bit 0 (IS_PU) is then set to indicate the sequence is complete.

Table 104. DBG_POW_STATE_SRAM0 Register

Bits	Description	Type	Reset
31:9	Reserved.	-	-
8	**IS_PU**: Indicates the power somain is fully powered up.	RO	0x0
7	**RESET_FROM_SEQ**: Indicates the state of the reset to the power domain.	RO	0x0
6	**ENAB_ACK**: Indicates the state of the enable to the power domain.	RO	0x0
5	**ISOLATE_FROM_SEQ**: Indicates the state of the isolation control to the power domain.	RO	0x0
4	**LARGE_ACK**: Indicates the state of the large power switches for the power domain.	RO	0x0
3	**SMALL_ACK2**: The small switches are split into 3 chains. In the power up sequence they are switched on separately to allow management of the VDD rise time. In the power down sequence they switch off simultaneously with the large power switches. This bit indicates the state of the last element in small power switch chain 2.	RO	0x0
2	**SMALL_ACK1**: This bit indicates the state of the last element in small power switch chain 1.	RO	0x0
1	**SMALL_ACK0**: This bit indicates the state of the last element in small power switch chain 0.	RO	0x0
0	**IS_PD**: Indicates the power somain is fully powered down.	RO	0x0

RP_AP: DBG_POW_STATE_SRAM1 Register

Offset: 0x014

Description

This register indicates the state of the power sequencer for the SRAM1 domain.
The sequencer timing is managed by the POWMAN_SEQ_* registers. See the header file for those registers for more information on the timing.
Power up of the domain commences by clearing bit 0 (IS_PD) then bits 1-8 are set in sequence. Bit 8 (IS_PU) indicates the sequence is complete.
Power down of the domain commences by clearing bit 8 (IS_PU) then bits 7-1 are cleared in sequence. Bit 0 (IS_PU) is then set to indicate the sequence is complete.

Table 105. DBG_POW_STATE_SRAM1 Register

Bits	Description	Type	Reset
31:9	Reserved.	-	-
8	**IS_PU**: Indicates the power somain is fully powered up.	RO	0x0
7	**RESET_FROM_SEQ**: Indicates the state of the reset to the power domain.	RO	0x0
6	**ENAB_ACK**: Indicates the state of the enable to the power domain.	RO	0x0
5	**ISOLATE_FROM_SEQ**: Indicates the state of the isolation control to the power domain.	RO	0x0
4	**LARGE_ACK**: Indicates the state of the large power switches for the power domain.	RO	0x0
3	**SMALL_ACK2**: The small switches are split into 3 chains. In the power up sequence they are switched on separately to allow management of the VDD rise time. In the power down sequence they switch off simultaneously with the large power switches. This bit indicates the state of the last element in small power switch chain 2.	RO	0x0
2	**SMALL_ACK1**: This bit indicates the state of the last element in small power switch chain 1.	RO	0x0
1	**SMALL_ACK0**: This bit indicates the state of the last element in small power switch chain 0.	RO	0x0
0	**IS_PD**: Indicates the power somain is fully powered down.	RO	0x0

RP_AP: DBG_POW_OVRD Register

Offset: 0x018

Description

This register allows external control of the power sequencer outputs for all the switched power domains. If any of the power sequencers stall at any stage then force power up operation of all domains by running this sequence:

- set DBG_POW_OVRD = 0x3b to force small power switches on, large power switches off, resets on and isolation on
- allow time for the domain power supplies to reach full rail
- set DBG_POW_OVRD = 0x3b to force large power switches on
- set DBG_POW_OVRD = 0x37 to remove isolation
- set DBG_POW_OVRD = 0x17 to remove resets

Table 106. DBG_POW_OVRD Register

Bits	Description	Type	Reset
31:7	Reserved.	-	-
6	**DBG_POW_RESTART_FROM_XOSC**: By default the system begins boot as soon as a clock is available from the ROSC, then it switches to the XOSC when it is available. This is done because the XOSC takes several ms to start up. If there is a problem with the ROSC then the default behaviour can be changed to not use the ROSC and wait for XOSC. However, this requires a mask change to modify the reset value of the Power Manager START_FROM_XOSC register. To allow experimentation the default can be temporarily changed by setting this register bit to 1. After setting this bit the core must be reset by a Coresight dprst or a rescue reset (see RESCUE_RESTART in the RP_AP_CTRL register above). A power-on reset, brown-out reset or RUN pin reset will reset this control and revert to the default behaviour.	RW	0x0